MASTER PROMPT — PROPERTY MANAGEMENT SYSTEM (MULTI-TENANT SAAS)

1. Your Role and Project Objective

Act as a senior Laravel developer, software architect, database designer, financial-systems analyst, security engineer, and UI/UX designer.

I want to build a complete, secure, practical, and beginner-friendly **Property Management System** for managing property purchases, sales, investors, expenses, payments, cash/bank accounts, profit/loss, staff, and financial reports.

The system should initially launch with free accounts and support paid subscription plans in the future.

My priority is a reliable, easy-to-use property business management application. Do not turn it into an unnecessarily complicated ERP, banking application, or enterprise accounting system.

Your responsibility is to understand all requirements below, design the database and application properly, and help build a working system with reliable financial calculations, secure tenant isolation, audit history, and a clean interface.

Important instructions

1. Do not ignore, silently change, or contradict the finalized business requirements below.
2. Do not introduce unnecessary modules or advanced features that are not requested.
3. Use sensible defaults for minor implementation decisions.
4. If a genuinely critical requirement is missing or two requirements conflict, ask a concise clarification before making an irreversible architectural decision.
5. Before writing application code, provide a concise implementation plan, proposed database structure, key relationships, and module sequence.
6. Then implement the project incrementally, ensuring each phase works before moving to the next.
7. Do not provide pseudo-code when working code is required.
8. Do not claim that code, migrations, tests, or features have been implemented or tested unless they actually have.
9. Explain setup and development instructions in simple language suitable for a beginner.

10. If working in an existing project, inspect its files, dependencies, database schema, and current implementation before modifying anything. Preserve working features unless a change is necessary and agreed.
11. When generating code, identify exact file paths and provide complete, usable code or precise patches. Include necessary migrations, models, controllers, routes, Blade views, validation, authorization, and tests.
12. Do not generate the entire application in one enormous, unverified code dump. Build it in manageable, testable phases.

---

2. Recommended Technology Stack

Use the following stack unless a genuine compatibility issue requires a justified alternative:

- Backend: Laravel, using the current stable version compatible with the selected PHP version at implementation time.
- Language: PHP.
- Database: MySQL.
- Frontend: Laravel Blade and Bootstrap 5.
- Authentication: Laravel-compatible authentication solution.
- Authorization: Laravel policies/gates and a suitable role/permission implementation.
- JavaScript: Minimal JavaScript and jQuery only where useful.
- Forms: Prefer standard Laravel forms and server-side validation over unnecessary AJAX.
- Reports: Excel/CSV and PDF export.
- Storage: Private file storage with authorized downloads.
- Testing: Laravel automated tests, including database, authorization, tenant isolation, and financial calculation tests.

Use Laravel conventions, service classes where useful, database transactions, foreign keys, indexes, policies, and clear separation of business logic.

Do not add microservices, Kubernetes, complex event-driven architecture, or other advanced infrastructure without a real need.

The initial application currency is **PKR only**.

The entire application interface must be in **English**. Menus, forms, buttons, validation messages, reports, and dashboard labels should use English. Urdu translation is not required initially.

---

3. Multi-Tenant Architecture — Critical Requirement

This is a multi-tenant SaaS-style application.

Each account holder must have one independent workspace.

Each workspace contains its own:

- Properties.
- Investors.
- Staff.
- Purchases and sales.
- Expenses.
- Payments and receipts.
- Cash and bank accounts.
- Investor allocations and distributions.
- Buyer receivables and seller payables.
- Documents.
- Approvals.
- Reports and audit history.

Strict tenant isolation

An account holder must never access another account holder's private business data.

Enforce tenant isolation in the backend through proper relationships, authorization policies, and scoped queries. Hiding a menu or button is not sufficient.

Check tenant ownership on every relevant request, including:

- Show, edit, update, cancel, and delete/soft-delete operations.
- File upload and download.
- Reports and exports.

- Financial transactions.
- Investor statements.
- Approval requests.
- Search, filters, and AJAX/dependent dropdown endpoints.

Prevent insecure direct object reference (IDOR) vulnerabilities.

All tenant-owned records must be linked to the correct workspace/account holder. Use a consistent tenant ownership strategy throughout the application.

Super Admin must not have silent, unrestricted access to customer business data. Exceptional support access must require authorization, a recorded reason, and an audit trail.

---

4. User Types and Access Control

A. Super Admin

Super Admin manages the overall platform, including:

- Customer account registrations and activation.
- Account suspension and reactivation.
- Account plans and feature permissions.
- Property and staff limits.
- Document upload/storage entitlements.
- Platform health and backup status.
- Future subscription management.

Super Admin must not silently view or change customer financial records. Exceptional support access must be authorized and logged.

B. Account Holder

Each account holder manages their own workspace and can:

- Add and manage properties.
- Manage investors.
- Manage property purchases and sales.

- Record expenses, receipts, and payments.
- Manage cash/bank accounts.
- Manage investor allocations and distributions.
- Manage staff and their permissions.
- Configure approval rules.
- View reports and export authorized data.
- Manage reservations, installments, and outstanding balances.
- Manage workspace settings and permitted features.

Account holders must not permanently delete investors or silently erase finalized financial history.

C. Staff

Account holders can create, update, and deactivate staff accounts.

Provide simple predefined roles, such as Admin and Staff, plus individual permission settings.

Permissions should cover the practical actions needed by the application, such as:

- View.
- Add.
- Edit.
- Cancel.
- Approve, where authorized.
- Export, where authorized.

Use backend authorization for every protected action.

Deactivating a staff member must revoke login access immediately while preserving their historical activity and records.

By default, staff must not approve their own approval requests.

D. Optional Investor Portal

An account holder may enable investor login for selected investors.

The investor portal is optional and may be restricted by the account's feature entitlement.

An investor may see only their own authorized investments, statements, relevant transactions, and profit distributions.

Investors must not see other investors' private details unless an explicit, justified business rule allows it.

Internal investor management must continue working even when investor portal access is disabled.

---

5. Super Admin Feature and Limits Management

Super Admin must be able to configure each customer account's plan and available features.

The system must support free accounts initially and future paid plans without requiring a major architectural rewrite.

Initially, subscriptions may be managed manually. A payment gateway is not required.

Configurable account entitlements

Support the following configurable features and limits:

1. Maximum active properties.
2. Maximum staff accounts.
3. Document upload permission.
4. Maximum document storage.
5. Maximum individual file size.
6. Permitted document/file types.
7. Excel/PDF export entitlement, if plan-restricted.
8. Investor portal entitlement.
9. Other future features through a maintainable entitlement mechanism.

The above limits are configurable examples, not predetermined commercial pricing or final numeric limits. Super Admin should be able to set appropriate values.

Active property counting

Count properties according to the configured definition of active inventory. Sold properties must not count as active inventory.

Do not count cancelled sale records as additional properties.

Storage restrictions

Document uploads may initially be enabled only for selected pilot accounts, such as one or two accounts, due to storage concerns.

When enabled, authorized users may attach relevant images and PDFs to:

- Purchase agreements.
- Sale agreements.
- Receipts.
- Expenses.
- Payments.
- Other supported business records.

Use private storage, secure downloads, file validation, storage quotas, and role-based authorization.

Plan changes and suspension

If an account is downgraded or its entitlement is removed:

- Preserve existing business data and documents.
- Do not automatically delete records.
- Apply configured restrictions to future actions where appropriate.
- Inform the account holder when an operation is blocked by a plan limit.

Account suspension must disable access as configured but must not erase financial history.

Design the entitlement mechanism so that new features can be enabled or restricted later.

---

6. Investor Management

Create a dedicated Investor CRUD/list-management area for each account holder.

An investor record should support appropriate fields, such as:

- Investor name.
- Contact details.

- CNIC/identification, if required by the business.
- Address, if required.
- Active/inactive status.
- Notes, if needed.
- Creation and modification history.

Avoid collecting unnecessary personal information.

Account holders can add and update investors, view their investments, and activate or deactivate them.

Investor deletion policy

Investors must not be permanently deleted.

Historical investments, transfers, settlements, statements, and transactions must remain linked to the investor.

An investor with an active property investment cannot be deactivated until their investment has been settled, transferred, or closed through an authorized process.

Investor deactivation must not erase historical records.

Investor portal

Allow the account holder to selectively enable investor login for individual investors, provided the account has the required entitlement.

---

7. Property Management

Create a property management module with a practical property list, create/edit forms, details page, filters, and status management.

Use a simple property organization model initially. Do not build a separate Projects/Blocks/Societies management module in the first version.

Suggested property fields

Include appropriate fields such as:

- Property reference/title.
- Optional project or society name as text.

- Block/sector/plot number.
- City and address.
- Property type.
- Size and size unit.
- Purchase date.
- Purchase price.
- Additional purchase expenses.
- Property status.
- Notes.

Support practical property types such as:

- Plot.
- House.
- Apartment.
- Shop.
- Commercial property.
- Agricultural land.
- Other configurable or extensible property types, if justified.

Allow search and filtering by property reference, project/society, location, type, and status.

Property lifecycle

Use a simple lifecycle:

Available → Reserved → Under Sale/Transfer Process → Sold

Allow a property to return to Available after a cancelled sale only when the relevant settlement and business rules permit it.

A cancelled sale must be preserved as a historical record. A subsequent sale must be a new sale record, not an overwrite of the cancelled sale.

Sold property history

When a property is sold:

- Mark it as Sold.
- Exclude it from active property inventory.
- Preserve the purchase, expense, investor, sale, payment, and profit/loss history.
- Keep it available in historical reports and investor statements.

Do not permanently delete property financial history.

Property valuation

The initial system must track purchase cost and realized profit/loss only.

Do not build market valuation, estimated current market prices, unrealized gain/loss, valuation history, or valuation forecasting in the initial version.

---

8. Property Purchase and Seller Payments

For each property purchase, track:

- Total purchase price.
- Additional purchase expenses.
- Seller payment history.
- Amount paid.
- Outstanding seller payable balance.
- Payment dates and amounts.
- Payment method and relevant cash/bank account.
- Optional receipt or document attachment when enabled.

Support partial payments and installments without unnecessarily complicated contract-management functionality.

If there are multiple sellers, provide seller-wise payable tracking where needed.

Critical financial rule

The property purchase price must be counted as property cost only once.

Subsequent payments to the seller settle the outstanding payable and reduce cash/bank balances. They must not add the same purchase cost again.

Purchase expenses must be recorded separately from payments that settle an existing payable.

---

9. Property Expenses and Funding Sources

Support expenses associated with purchasing, owning, maintaining, or selling a property.

Examples include:

- Purchase-related expenses.
- Documentation and transfer charges.
- Repairs and maintenance.
- Agent/broker commission.
- Taxes and government charges.
- Other business or property expenses.

Expense categories should be manageable and extensible without needing a code change for every new category.

Separate expense recognition from payment

The system must distinguish:

- Expense incurred.
- Amount paid.
- Outstanding payable, if any.

Record relevant details, including amount, date, category, property, payer/funding source, payment method, and status.

An expense may be funded by:

- The account holder.
- One or more investors.
- A combination of the account holder and investors.

Record both:

1. Who actually paid the amount.
2. Whose funds or capital the expense represents.

These concepts must not be confused.

Avoid double-counting an expense when its payable is later paid.

---

10. Sale Management

A sale record should support:

- Property.
- Buyer details.
- Sale agreement date.
- Agreed sale price.
- Advance/token amount.
- Payment schedule, if applicable.
- Sale expenses.
- Amount received.
- Remaining buyer receivable.
- Sale status.
- Finalization date.
- Relevant documents, if enabled.
- Cancellation details, where applicable.

Sale finalization and profit/loss

When a sale is finalized, calculate actual realized profit/loss using the relevant recorded purchase cost, applicable purchase expenses, and relevant sale expenses.

A simplified conceptual calculation is:

Net realized profit/loss = Sale price − allocated property cost − applicable sale expenses.

The precise cost and expense allocation must follow consistent, documented business rules and must not count the same expense twice.

Keep profit separate from cash receipts

A sale may be finalized even when the buyer has not paid the entire amount.

Track separately:

- Finalized sale value.
- Calculated realized profit/loss.
- Cash actually received.
- Remaining buyer receivable.
- Investor profit payable.
- Investor profit actually paid.

Do not treat outstanding buyer installments as cash received.

Do not treat an investor profit distribution as a new property expense.

---

11. Sale Expenses, Commission, and Taxes

Support configurable sale expense categories, including:

- Agent/broker commission.
- Transfer charges.
- Documentation expenses.
- Taxes and government fees.
- Other sale-related expenses.

Record category, amount, date, payment method/account, payment status, and any optional attachment.

Applicable sale expenses must be included in net realized profit/loss according to consistent business rules.

Do not build a complex tax engine, automatic tax-law updates, or advanced withholding-tax calculations in the initial version.

Tax and expense categories may be recorded without implementing an advanced tax-compliance module.

---

12. Buyer Reservations, Installments, and Late Payments

Support property reservations and flexible buyer installment plans.

Reservation

A reservation may include:

- Buyer details.
- Reservation date.
- Token/advance amount.
- Expiry date.
- Outstanding amount.
- Receipt, where attachments are enabled.
- Cancellation and refund details.

A token or advance is not automatically profit. Treat it as a buyer advance/liability until the sale is finalized and apply the correct accounting treatment.

Installment management

Support:

- Custom installment schedules.
- Due dates and amounts.
- Partial payments.
- Advance payments.
- Payment history.
- Remaining receivable.
- Overdue status.
- Due-date and overdue reminders.
- Optional late fees, only where the account holder has configured them and the relevant agreement permits them.

Allow practical schedule adjustments without introducing unnecessary complex penalty engines or default-risk analytics.

When a sale is cancelled, track refunds, remaining refunds, permitted forfeiture/cancellation charges, and outstanding amounts according to the agreement and applicable business rules.

A cancelled sale must remain in history.

---

13. Multiple Cash and Bank Accounts

The purpose of this module is to track the property's business money. It is not intended to be a complete banking application.

Allow an account holder to create multiple cash accounts and bank accounts.

Each account should track:

- Account name/type.
- Opening balance.
- Receipts.
- Payments/withdrawals.
- Transfers.
- Current balance.

Record relevant transactions with dates, amounts, descriptions, and references to the related property, purchase, sale, expense, or investor transaction where applicable.

Transfers

Allow money transfers between the account holder's cash/bank accounts.

A transfer must:

- Reduce the source account balance.
- Increase the destination account balance.
- Be recorded as one linked transfer or equivalent balanced entries.
- Not be counted as income or expense.

Avoid double-counting

Receiving cash from a buyer and later depositing that cash into a bank account must not count as two separate sales or two separate income events.

Likewise, withdrawing money from a bank account into cash must not count as a new expense.

Do not implement bank statement imports, bank reconciliation, or advanced treasury management in the initial version.

---

14. Investor Allocations and Investment Ratios

A property may have multiple investors.

Track:

- Investor.
- Property.
- Investment contribution.
- Contribution date.
- Relevant capital allocation.
- Ownership percentage.
- Profit/loss sharing percentage.
- Investment status and history.

Allocation cap

Total investor capital allocations must not exceed the property's total purchase cost, defined as purchase price plus purchase-related expenses.

Any remaining amount is treated as the account holder's own investment.

Do not allow allocations to silently exceed this cap. If later changes to property costs affect the calculation, handle them through a validated process.

Separate ratios

Ownership percentages and profit/loss sharing percentages are separate concepts.

- Ownership percentages default to the investment ratio but may be configured according to the agreement.
- Profit/loss sharing percentages default to the investment ratio but may be configured separately.

- Validate that the relevant active percentages total 100% for the applicable property or allocation group.
- Preserve the applicable percentages and agreement history for finalized transactions.

Do not assume that ownership and profit-sharing ratios are always identical.

---

15. Investor Changes, Transfers, and Exit

Before sale finalization, authorized users may:

- Add an investor.
- Transfer part or all of an investor's share to another investor.
- Allow an investor to exit.
- Replace an investor.

Record relevant details:

- Effective date.
- Previous and new investor.
- Transferred percentage or capital amount.
- Agreed valuation, where applicable.
- Settlement amount.
- Payment status.
- Reason and audit history.

An investor-to-investor share transfer is not a property sale and must not be recorded as property sale income.

Preserve historical investor allocations and agreements.

After a property sale is finalized, preserve the applicable investor ratios used for that sale. Corrections must use the dedicated audited correction process.

---

16. Investor Profit Distribution

When a property sale is finalized:

1. Calculate total realized profit/loss.
2. Calculate each investor's applicable share according to the recorded profit-sharing percentages.
3. Record the investor's profit payable or loss allocation as applicable.
4. Track actual distributions separately from the calculated payable.

Maintain separate values for:

- Calculated investor profit share.
- Profit payable.
- Amount paid.
- Remaining unpaid amount.
- Payment date and method.
- Related cash/bank account.
- Relevant payment reference.

The account holder decides when to pay investors, subject to the applicable agreements and approval rules.

Available cash and buyer installment receipts must not be confused with calculated profit.

Investor payouts must not be counted again as property expenses.

---

17. Financial Ledger and Integrity

Implement a practical, auditable financial ledger suitable for property business operations and capable of supporting a more formal double-entry accounting system in the future.

Do not build a full enterprise accounting package initially.

Track the necessary financial concepts separately:

- Cash and bank balances.
- Buyer receivables.
- Seller payables.
- Property purchase costs.

- Expenses incurred.
- Expense payments.
- Investor capital.
- Investor profit payable.
- Actual investor distributions.
- Buyer advances and refunds.
- Sale proceeds and realized profit/loss.
- Account-to-account transfers.

Use consistent financial services or calculation classes rather than duplicating calculation logic across controllers and views.

Financial precision

Use appropriate fixed-precision decimal database columns for PKR amounts, not floating-point values for financial calculations.

Use database transactions for multi-step financial operations.

Use appropriate validation, constraints, and locking strategies where needed to prevent duplicate payments, invalid balances, and inconsistent concurrent updates.

Never calculate a financial report by blindly adding all payment records together without considering the type and accounting meaning of each transaction.

---

18. Corrections, Cancellations, and Audit History

Draft transactions

Draft transactions may be edited or deleted according to permissions and business rules.

Confirmed transactions

Confirmed financial transactions must not be silently overwritten or permanently deleted.

Corrections must use a reasoned, audited adjustment process that preserves:

- Original values.
- Corrected values.

- Reason for correction.
- User making the correction.
- Date/time.
- Relevant approval history, where applicable.

If an actual payment is reversed, record a separate reversal entry linked to the original payment.

Finalized sale corrections

An authorized account holder may correct a finalized sale with a mandatory reason and audit trail. Super Admin approval is not required by default.

Preserve the original sale and correction history. Recalculate applicable balances and profit/loss consistently.

Do not overwrite payments already made to investors. If a correction creates an additional payable or recoverable amount, record it separately.

Sale cancellation

Sale cancellation must retain the historical sale record.

Record cancellation date/reason, received payments, refunds, outstanding refunds, permitted forfeitures or cancellation charges, and other settlement amounts.

A cancelled sale is not automatically erased from the property's history.

---

19. Approval Workflow

Provide a simple configurable approval workflow for selected actions.

Account holders can decide which of the following actions require approval:

- Property purchase.
- Sale finalization.
- Large expenses.
- Refunds.
- Investor payouts.

Allow sensible configurable thresholds where appropriate.

Use statuses:

- Draft.
- Pending Approval.
- Approved.
- Rejected.

Record requester, approver, timestamps, reasons, and relevant audit history.

Staff must not approve their own requests by default.

When approval is required, do not allow the financial operation to be finalized before approval.

Notifications may inform users about approval requests and decisions, but notifications themselves must not create financial transactions.

Do not force approvals on every routine entry unless the account holder configures that requirement.

---

20. Dashboard and Financial Reports

Create a clear dashboard showing practical business KPIs, such as:

- Total active properties.
- Available, reserved, under-sale, and sold properties.
- Total recorded purchase costs.
- Actual realized profit/loss.
- Cash/bank balances.
- Buyer receivables.
- Seller payables.
- Outstanding buyer installments.
- Investor profit payable and unpaid distributions.

Clearly distinguish accounting/realized profit from cash received.

Required reports

1. Property list and status report.

2. Property-wise financial ledger.
3. Purchase and seller payment report.
4. Sale and buyer receivable report.
5. Expense report.
6. Realized profit/loss report.
7. Investor investment statement.
8. Investor profit distribution statement.
9. Cash/bank summary.
10. Seller outstanding/payable report.
11. Buyer overdue/installment report.
12. Date-wise financial reports.

Provide date-range filters, search, pagination where appropriate, and Excel/PDF export.

All reports and exports must enforce tenant isolation and staff permissions.

Do not build advanced forecasting, market-value analysis, or a custom report builder in the initial version.

---

21. Documents and Attachments

Document functionality is optional and must be controlled through account entitlements.

Initially, Super Admin may enable uploads only for selected pilot accounts.

When enabled, support categorized documents, multiple files, upload dates, notes, and relevant links to:

- Properties.
- Purchases.
- Sales.
- Expenses.
- Payments.
- Other relevant supported records.

Support images and PDFs initially.

Purchase agreements and sale agreements must have an attachment option when the account has the required entitlement.

Use:

- Private storage.
- File type and size validation.
- Configurable storage quotas.
- Role-based access.
- Secure authorized download routes.
- Appropriate version/history tracking where documents are replaced.
- Audit logging for sensitive document actions.

Never expose private files through predictable public URLs.

If an account loses its upload entitlement, preserve existing files and restrict future uploads according to the plan rules.

---

22. Notifications and Reminders

Implement useful in-app notifications and configurable email notifications where practical.

Support:

- Approval requests and decisions.
- Buyer installment due dates.
- Overdue installments.
- Relevant outstanding payment reminders.
- Important account/security events.
- Backup failure alerts for authorized administrators.

Allow user-wise notification preferences where useful.

Staff must see only notifications they are authorized to access.

Avoid duplicate reminders and record email delivery failures where supported.

Notifications must not independently create or modify financial transactions.

---

23. Registration and Account Activation

Provide a public signup request process for new account holders.

The intended flow is:

1. User submits registration details.
2. User verifies their email.
3. The account awaits Super Admin approval/activation.
4. Super Admin approves or rejects the request.
5. After approval, the account holder can access their workspace.

Support account activation, suspension, and reactivation.

Initially, no payment gateway or automated paid subscription billing is required.

Super Admin may assign the free plan and configure entitlements manually.

Account suspension must not delete business data.

---

24. Security and Login

Use secure authentication with:

- Email and password.
- Password reset.
- Email verification.
- Appropriate password policies.
- Login rate limiting.
- Secure sessions.
- Logout.
- Optional two-factor authentication for account holders and Super Admins.
- Login/logout activity logging.

- Appropriate suspicious-login protection.

Use CSRF protection, output escaping, secure validation, and authorization on every sensitive backend operation.

Protect against SQL injection, mass assignment, insecure file uploads, IDOR, cross-tenant data leaks, and unauthorized financial changes.

Never store plain-text passwords or expose credentials in logs.

---

25. Backup and Recovery

Implement automatic daily database backups.

Configure retention to a reasonable period based on hosting capacity and the business's needs.

Provide backup status monitoring and alerts when backups fail.

When document storage is enabled, include the relevant uploaded files in the backup strategy.

Prefer encrypted off-site copies where practical and supported by the hosting environment.

Implement and test a restoration procedure that maintains consistency between database records and document files.

Restrict backup and restore operations to authorized administrators.

Do not claim backups are working until the scheduled process and restoration procedure have actually been tested.

---

26. Hosting and Deployment

The application will run on secure online web hosting compatible with Laravel and MySQL.

Use HTTPS/SSL.

The deployment must support:

- Application configuration through environment variables.
- Secure database credentials.
- Laravel queues or scheduled jobs only where needed.
- Daily backup scheduling.

- Private file storage.
- Logs and basic error monitoring.
- Proper production cache and permissions.
- A reliable migration and deployment process.

Do not expose .env, database backups, private documents, logs, or internal application files publicly.

Advanced cloud infrastructure, Kubernetes, and complex DevOps automation are not required initially.

Provide deployment instructions suitable for a beginner, including hosting requirements, database setup, environment configuration, migrations, storage, scheduler/cron configuration, and backup verification.

---

27. Data Export and Account Closure

Provide structured exports of authorized business data, including relevant Excel/CSV records, financial statements, and available documents.

Enforce tenant isolation and permissions on all exports.

Log sensitive export operations.

Account closure must disable login while preserving financial records and audit history according to the applicable retention policy.

Deletion requests must be reviewed against applicable legal, financial, and business record-retention obligations.

Do not automatically erase historical transactions when an account closes.

---

28. User Interface and Ease of Use

The application is intended for practical property business users, not technical accountants.

Use a clean, professional, responsive Bootstrap 5 interface.

Provide:

- Consistent navigation.

- Clear forms and labels.
- Useful validation messages.
- Confirmation screens for sensitive actions.
- Clear financial totals.
- Search and filters.
- Pagination for long lists.
- Status indicators.
- Useful empty states.
- Human-readable dates and PKR formatting.
- Responsive layouts for desktop and mobile.

Keep forms understandable and avoid unnecessary fields.

Use normal Laravel form submissions where practical.

Use JavaScript only when it materially improves the experience, such as dependent dropdowns or dynamic line items.

Do not expose database column names or internal technical terms to end users unnecessarily.

---

29. Database Design Requirements

Design a normalized, maintainable relational schema.

Likely entities will include:

- Users.
- Tenant/workspace accounts.
- Account plans and entitlements.
- Staff permissions.
- Investors.
- Properties.
- Purchases.

- Purchase expenses.
- Sellers and seller payments, where needed.
- Sales.
- Buyers and buyer payments.
- Reservations.
- Installment schedules.
- Expenses and expense payments.
- Cash/bank accounts.
- Financial transactions and account transfers.
- Property investor allocations.
- Ownership/profit-sharing agreements and their history.
- Investor share transfers and settlements.
- Investor profit payables and distributions.
- Approval requests.
- Documents and document versions.
- Notifications.
- Audit logs.
- Backup status or related operational metadata, where appropriate.

These are suggested logical entities, not a requirement to create a separate table for every item. Choose the simplest normalized design that preserves the required history and financial integrity.

Before implementation:

1. Present the proposed database structure.
2. Explain important relationships and tenant ownership.
3. Identify financial records that must be immutable or correction-based.
4. Explain how purchase cost, expenses, payments, receivables, payables, and investor distributions avoid double-counting.

5. Confirm that all necessary historical data can be preserved.

Use foreign keys, indexes, appropriate deletion policies, and database constraints.

Use soft deletes where suitable for business records, but do not rely on soft deletes as a substitute for financial audit history.

---

30. Required Automated Tests

Write and run tests for critical business rules.

At minimum, test:

Tenant isolation

- One account holder cannot access another account's properties, investors, transactions, documents, or reports.
- Staff access is restricted to their workspace and permissions.
- Investor portal users can see only their own authorized information.

Property and investor rules

- Sold properties are not counted as active inventory.
- Investor allocations cannot exceed the defined capital allocation cap.
- Ownership percentages and profit-sharing percentages are validated separately.
- Investor deactivation is blocked when an active investment remains unsettled.

Financial integrity

- Purchase cost is counted once.
- Seller installment payments do not duplicate purchase cost.
- Expense recognition and expense payment are not double-counted.
- Account-to-account transfers do not count as income or expense.
- Buyer advances are not automatically treated as profit.
- Buyer receivables and actual cash receipts remain separate.
- Investor distributions do not count as new property expenses.
- Investor profit payable and profit paid remain separate.

- Cancelled sales retain history and record refunds/settlements.
- Sale corrections preserve original history and adjust balances consistently.
- Reversals create separate linked entries.
- Concurrent or duplicate payment submissions cannot silently duplicate financial transactions.

Authorization and entitlements

- Disabled staff cannot log in.
- Approval requirements are enforced.
- Unauthorized users cannot approve restricted operations.
- Account property limits are enforced.
- File uploads and downloads respect entitlements and permissions.
- Plan downgrade does not delete existing business data.

Backup and reports

- Financial reports use correct date filters and calculations.
- Exports respect tenant isolation.
- Backup and restoration procedures can be verified.

Do not mark tests as passed unless they have actually been executed successfully.

---

31. Recommended Development Phases

Develop the project in a sensible order.

Phase 1 — Project Foundation

- Laravel setup.
- Database configuration.
- Authentication.
- Tenant/workspace model.
- Super Admin and account-holder roles.

- Basic authorization.
- English interface layout.

Phase 2 — Account Management and Entitlements

- Signup requests.
- Email verification.
- Super Admin activation.
- Account status management.
- Feature and limit management.
- Free-plan defaults.

Phase 3 — Investors, Staff, and Properties

- Investor management.
- Staff management and permissions.
- Property management.
- Property status lifecycle.
- Active property counting.

Phase 4 — Purchases and Expenses

- Property purchases.
- Seller records.
- Purchase expenses.
- Seller payments and outstanding balances.
- Expense management and funding sources.

Phase 5 — Cash/Bank and Financial Records

- Multiple cash/bank accounts.
- Receipts and payments.
- Transfers.
- Consistent ledger and balance calculations.

Phase 6 — Sales and Buyer Installments

- Reservations.
- Sale records.
- Buyer advances and receipts.
- Installment schedules.
- Sale expenses.
- Cancellation and refunds.
- Sale finalization and profit/loss.

Phase 7 — Investor Allocations and Distributions

- Investor capital allocation.
- Ownership and profit-sharing ratios.
- Investor changes and transfers.
- Investor payable calculations.
- Profit distributions and settlements.

Phase 8 — Approvals, Documents, and Notifications

- Configurable approvals.
- Feature-gated document uploads.
- In-app/email notifications.
- Due-date and overdue reminders.

Phase 9 — Dashboard and Reports

- Dashboard KPIs.
- Financial reports.
- Investor statements.
- Cash/bank summaries.
- Outstanding buyer/seller balances.
- Excel/PDF exports.

Phase 10 — Security, Testing, and Deployment

- Comprehensive authorization tests.
- Tenant-isolation tests.
- Financial calculation tests.
- Backup and restore verification.
- Production deployment.
- Final documentation.

You may adjust the order when dependencies justify it, but explain any significant change.

---

32. Implementation Workflow and Communication

I am not an expert developer, so guide me step by step.

For each development phase:

1. Explain the objective in simple English.
2. Identify the files and database changes involved.
3. Provide the exact commands and file paths.
4. Provide complete usable code or precise patches.
5. Explain where the code belongs.
6. Include migrations, relationships, validation, authorization, and routes as required.
7. Tell me how to run migrations and tests.
8. Provide practical manual testing steps.
9. Wait for my confirmation or test results before moving to the next major phase when working interactively.
10. If an error occurs, investigate the actual error and provide a focused fix rather than rebuilding unrelated modules.

Never invent existing files, routes, database fields, test results, or environment details.

If I share an existing code file, inspect it and preserve relevant existing functionality.

Do not change the established business rules merely to make implementation easier. If a rule is technically difficult, explain the issue and propose the simplest correct solution.

Avoid unnecessary repetition, oversized controllers, duplicated calculation logic, and unexplained code.

---

33. Final Acceptance Criteria

The project should not be considered complete merely because its screens exist.

It must have a working end-to-end flow in which:

1. Super Admin activates an account and assigns its entitlements.
2. The account holder creates investors and properties.
3. The account holder records purchase costs, additional expenses, investor allocations, and seller payments.
4. The account holder records cash/bank accounts and transactions.
5. A property can be reserved and sold.
6. Buyer installments, receipts, outstanding balances, and sale expenses are tracked.
7. Finalized sale profit/loss is calculated correctly.
8. Investor shares, profit payables, and actual distributions are tracked separately.
9. Transfers, corrections, cancellations, and refunds preserve financial integrity.
10. Staff permissions and configurable approvals work on the backend.
11. Account-specific feature limits are enforced without deleting historical data.
12. Reports and exports show correct, tenant-isolated financial data.
13. Backup and restoration procedures are verified.
14. Security, tenant isolation, and critical financial tests pass.

---

34. Your First Response to This Prompt

Do not immediately generate the entire application.

First, provide:

1. A concise summary confirming your understanding of the system.
2. A proposed high-level architecture.
3. A proposed database/entity relationship plan.
4. A short explanation of how the financial model avoids double-counting.
5. A development phase checklist.
6. Any genuinely critical contradictions or missing decisions that must be resolved before coding.

Keep this first response focused. Do not reopen finalized decisions or ask questions already answered in this prompt.

Once the plan is reviewed, begin Phase 1 and implement the project incrementally with working code and tests.

The overriding objective is a secure, financially reliable, easy-to-use property management system that solves the actual business needs without unnecessary complexity.